

Assignment: Mobile Application Development Fundamentals

1. Differences between Native, Hybrid, and Cross-platform mobile application development

When people talk about **native development**, they mean building an app using the actual language and tools that a platform was designed for: Swift or Objective-C on iOS, Kotlin or Java on Android. This gives you the smoothest performance and full access to whatever the device can do, since there's nothing sitting between your code and the hardware. The catch is you basically have to build the app twice: once for each platform. That's why native is usually the go-to for apps where performance really matters, like games, camera-heavy tools, or anything using AR/VR.

Hybrid development is more or less a website wrapped inside an app shell, built with HTML, CSS, and JavaScript, then packaged using something like Apache Cordova or Ionic. Since it's one codebase running everywhere, it's quicker and cheaper to put together, but it tends to feel a bit slower and can't tap into every device feature the way a native app can. You'll usually see this approach in simpler apps, content-driven apps, internal tools a company uses in-house, or an MVP that just needs to get out the door fast.

Cross-platform development sits somewhere in the middle. You write the app once, in something like Dart for Flutter or JavaScript for React Native, and it gets compiled down into actual native components for each platform. That means you get performance and a look-and-feel that's close to native, without the cost of maintaining two completely separate codebases. This is why most commercial apps today lean toward cross-platform: teams want to launch on Android and iOS at the same time without blowing the budget.

2. The Flutter Ecosystem and how it helps developers create modern mobile applications

The Flutter Ecosystem is basically everything Google built around Flutter to make app development easier: not just the framework itself. It's built on the Dart language and comes with the Flutter SDK, a huge library of ready-made widgets, the rendering engine that draws the UI directly (Skia,

or Impeller on newer versions), plus tools like the Flutter CLI and DevTools for debugging and checking performance. On top of that, there's pub.dev, a repository with thousands of packages other developers have already built and shared. What makes this useful in practice is that you write your app once and it runs on Android, iOS, web, and even desktop, so you're not duplicating work across platforms. The widget system makes it pretty straightforward to design a custom looking UI instead of being stuck with generic components, and hot reload means you can tweak your code and see the result almost instantly instead of waiting for a full rebuild. Because Flutter draws its own UI rather than relying on whatever the OS provides, the app looks and behaves the same no matter what device it's on. And with pub.dev, you rarely have to build common stuff like authentication or networking from zero someone's probably already published a package for it.

3. The Software Development Life Cycle (SDLC) and its importance

The SDLC is just a structured way of getting a piece of software from idea to finished product. It's usually broken into phases figuring out requirements, planning, designing, actually coding, testing, deploying, and then maintaining the app once it's live. Each phase feeds into the next one instead of everything happening at once in a rush. It matters because without some kind of process, projects tend to fall apart requirements get misunderstood, bugs slip through because there was no real testing phase, deadlines get blown past, and the final product doesn't actually match what users needed. Following the SDLC keeps a team on the same page, catches problems earlier when they're cheaper to fix, and generally leads to software that's more reliable and easier to maintain down the line.

4. How Flutter development practices and the SDLC work together in real-world applications

In practice, Flutter doesn't replace the SDLC it just fits inside it. During the planning and design phase, developers are already thinking about how the app's screens and navigation will be structured in Flutter, and deciding on a state management approach like Provider, Riverpod, or Bloc. Once it's time to actually build the app, hot reload and Flutter's widget library make the coding phase go a lot faster since you're constantly testing small changes as you go instead of waiting around. When it comes to testing, Flutter has built-in support for unit tests, widget tests, and integration tests, which lines up directly with the testing stage of the SDLC. For

deployment, the same codebase gets compiled into an Android build and an iOS build, which makes the release stage a lot less painful than maintaining two separate apps. And afterward, during maintenance, Flutter's widget-based structure and its package ecosystem make it easier to patch bugs or add new features without rewriting everything. Basically, the SDLC gives the project its overall roadmap, and Flutter is the toolset that makes moving through each stage faster and less painful.

5. Why developers should understand these concepts before starting an application project

Knowing the difference between native, hybrid, and cross-platform means you can actually pick the right approach for the project instead of guessing based on budget, timeline, and how much performance you really need instead of realizing halfway through that you picked the wrong one. Understanding the Flutter Ecosystem means you're not reinventing things that already exist. You can pull from existing packages and tools instead of building everything from scratch. Understanding the SDLC keeps the whole project organized proper planning, proper testing, proper maintenance which lowers the chances of the project falling apart or missing what the users actually wanted. Put together, these three things give a developer a real foundation before they even start coding. Knowing what you're building, how you're going to build it, and the process you'll follow to get there makes it a lot more likely the app actually ships on time, on budget, and matches what people needed in the first place.